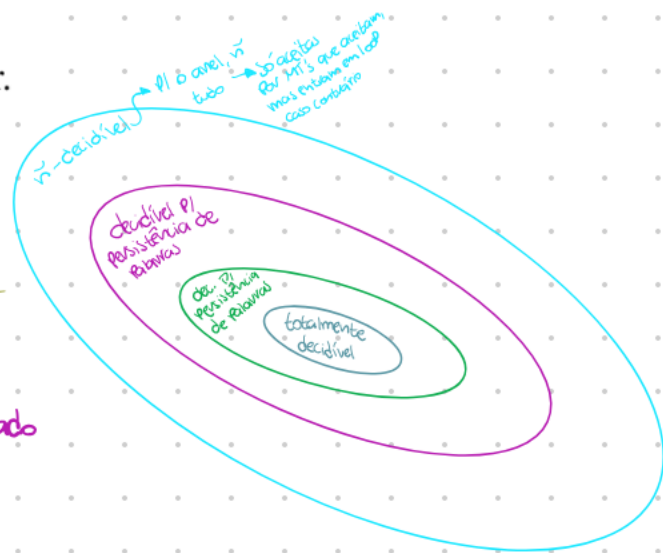


1. Você deve saber:

- Caracterizar linguagens e associá-las ao melhor mecanismo reconhecedor.

• Hierarquia de Chomsky

tipo	Linguagem	Gramática	Reconhecedor
3	regular	regular	autômato finito
2	livre de contexto	livre de contexto	autômato c/ pilha
1	Sensível ao contexto	Sensível ao contexto	autômato linearmente limitado
0	recursivamente enumerável	irrestrita	máquina de Turing



- n conta infinitamente, apenas 'lembra' em qual estado está
- validação de padrões fixos, repetições simples ou contagens finitas (n var de a's)
- n compara contagens 2 variáveis independentes
- $a^n b^n$ n é, Pq. Precisa contar qnt 'a's' para depois ver se tem a msm de 'b's'

- consegue empilhar p/ lembrar a qntd e desempilhar p/ comparar
- faz uma comparação direta de contagem ou inversão de ordem
- só olha o topo; n mantém 2 contagens independentes

→ p/ saber se é LLC: $a^n b^n$ n é

• se fecham c/ parênteses $a^i b^j c^k d^l$ $i=l$ e $j=k$

• se tem 'ou' → n -determinismo

- lê e escreve numa fita, mas o tamanho é limitado ao da entrada
- +1 comparação ou dependências cruzadas → $a^n b^n c^n$

- lê e escreve sem limite de tamanho
- tudo que é computável por um algoritmo moderno

- As definições formais de AF, AP e MT.

• AF: $M = (Q, \Sigma, \delta, q_0, F)$

alfabeto entrada, estado inicial, estados finais, função transição

$[q_0, aab] \vdash [q_1, ab] \vdash [q_2, b] \vdash [q_3, \epsilon]$

• AP: $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$

alfabeto da pilha

$[q_0, aaaa, \epsilon]$
 $[q_1, aaaa]$

• MT: $M = (Q, \Sigma, \Gamma, \delta, q_0, L, F)$

alfabeto da fita

$q_0 a b \vdash x q_1 b \vdash x y q_2 a$

aceitação

$[q_1, aaaa, \$]$ $[q_1, aaa, a \$]$ $[q_2, aaa, a \$]$ $[q_1, aa, a \$]$ $[q_1, a, aaaa \$]$ $[q_2, aa, aa \$]$ $[q_1, a, \$]$ $[q_1, \epsilon, aaaa \$]$ $[q_2, a, aaa \$]$ $[q_2, a, a \$]$ $[q_2, \epsilon, aaaa \$]$ $[q_2, \epsilon, aa \$]$ $[q_2, \epsilon, \$]$ $[q_3, \epsilon, \epsilon]$ ✓

- Conhecer as principais extensões de MT (multifitas e não determinística)

• todas MT são equivalentes, variações só facilitam

• multifitas:

- 1ª fita c/ entrada
- outras com espaço vazio

• $\bar{n}$ -det:

- msm ideia básica, mas $\bar{n}$ aumenta poder como no PDA

- Compreender a demonstração da existência de linguagens indecidíveis e não computáveis.

• $\bar{n}$ -computável: o conjunto de todas as linguagens possíveis é um infinito $\bar{n}$ -enumerável, mas o conjunto de MT's é infinitamente numerável, logo existem + linguagens do que MT's capazes de reconhecer-las

• $\bar{n}$ -decidível: diagonalização de Cantor gera uma contradição lógica quando tenta criar um algoritmo universal

OLHAR FOTO

2. O que é um conjunto enumerável? Qual é a relação entre enumerabilidade e computabilidade?

• é enumerável se for finito ou se tiver exatamente o mesmo "tamanho" que $\mathbb{N}$; dizer que um conjunto infinito é enumerável significa que, teoricamente, você consegue criar uma lista ordenada p/ seus elementos, de modo que cada elemento do conjunto tenha um n° natural correspondente

↳ n° s reais entre 0 e 1 $\bar{n}$ são enumeráveis

• a computabilidade só existe dentro do universo do que é enumerável

• como o universo das linguagens é $\bar{n}$ -enumerável, a enumerabilidade define o limite matemático do que é computável

3. Quais são as propriedades de um procedimento algorítmico? Relacione cada uma das propriedades com a definição da máquina de Turing e seus processos de computação.

• Finitude: um algoritmo deve sempre consistir em um n° finito de passos executáveis e deve, eventualmente, terminar p/ as entradas válidas, gerando uma resposta

↳ MT é baseada no conjunto finito de estados Q , e nos alfabetos finitos Σ e Γ ; a computação de uma MT que decide uma linguagem garante a finitude, pois a máquina é desenhada p/ alcançar um estado de parada em um n° finito de transições, sem ficar presa num loop p/ entradas que pertencem à linguagem

• ausência de ambiguidade: cada passo do algoritmo deve ser definido, $\bar{n}$ pode haver margem p/ interpretações diferentes

↳ na δ da MT $Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$, o que significa que dado o estado atual e o símbolo na fita, há uma única ação possível pela regra

• entradas e saídas bem definidas: um algoritmo opera sobre um conjunto de dados iniciais e processa esses dados p/ produzir um resultado final claro

↳ entrada: gravada na fita, máquina inicia apontando p/ o 1º símbolo

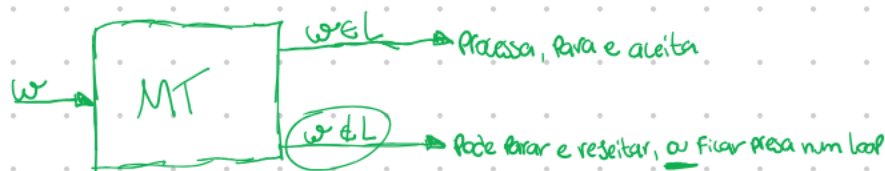
↳ saída: pode ser o estado final (reconhecida/descartada) ou o texto final após cessar suas transições (computadora de funções)

• efetividade: todas as operações devem ser básicas o suficiente p/ que possam, em princípio, ser feitas de forma exata por um humano usando papel e caneta em um tempo finito

↳ operações atômicas da cabeça de leitura e escrita da MT são efetivas; em um passo a máquina pode ler/escrever, mudar estado de controle e andar uma posição; nenhum passo exige processamento complexo

5. Defina Linguagem Recursiva e Linguagem Recursivamente Enumerável. Correlacione com as máquinas de Turing.

• uma linguagem é recursivamente enumerável se existe uma MT que a reconhece.



• uma linguagem é recursiva/decidível se existe uma MT que a decide.



• toda recursiva também é LRE, o oposto não é verdadeiro.

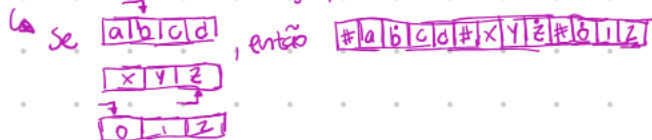
• o problema da parada é LRE, mas não é recursivo.

6. Apresente uma demonstração da equivalência entre Máquinas de Turing Multifitas e Máquinas de Turing com fita única.

• Projetar uma fita única que representa o conteúdo das k fitas ao mesmo tempo.

• Se $k=3$, a fita única terá 3 partes separadas por #s

• Para saber onde estão as cabeças, é criada uma cópia de cada símbolo c com um ponto em cima.



• na simulação: • varredura para identificar símbolos c com ponto em cima (ida)

- verifica δ da MTm e descobre:
 - ▲ quais novos símbolos são escritos em cada fita.
 - ▲ qual direção cada cabeça se move.
 - ▲ qual o próximo estado.

• varredura para atualizar modificações (volta).

• Se uma cabeça se mover para uma #, a MT pausa a simulação temporariamente, empurra toda a fita uma casa para a direita, abrindo um espaço vazio e colocando o ponto no novo espaço vazio; como MT é infinita à direita, esse empurrão sempre é possível.

7. Apresente uma demonstração da equivalência entre Máquinas de Turing Não Determinísticas e Máquinas de Turing Determinísticas.

• em uma MTN, a computação de uma palavra é uma árvore de possibilidades; para MTN aceitar, basta que um ramo aceite.

• Se MTD tentar simular essa árvore em um único ramo, pode entrar num loop infinito no ramo errado e não testar o de aceitação.

↳ deve simular numa busca em largura (todos caminhos de nível 1, 2, 3...), e, se houver caminho de aceitação, uma hora o MTD o alcança.

• MTD com 3 fitas (equivalente de uma única fita)

↳ • 1: entrada - nunca alterada

• 2: copia entrada e faz simulação do caminho atual

• 3: guarda a 'foto' ou o 'endereço' do nó da árvore que a máquina está explorando no momento

↳ se endereço for 213 → no primeiro passo, pegue a segunda transição; no segundo passo, pegue a primeira transição...

↳ • Para cada passo '3' lido, é executada a transição na '2'

▲ aceita: MTD para e aceita

▲ termina/aborta: se endereço em '3' é inválido (pediu 3ª escolha, mas o estado só tinha 2 escolhas) ou se chegou a uma redefinição, a máquina aborta esse passo

■ incrementa o endereço em '3' lexicograficamente (213 → 214 - ou o próximo nível da árvore); limpa '2', copia a entrada original e faz a simulação novamente

8. Reconstrua com suas próprias palavras a Prova do Problema da Parada. Relacione esta prova com a diagonalização de Cantor.

8 O objetivo do halting problem é provar que é matematicamente impossível criar um algoritmo genérico que consiga olhar p/ qualquer programa e prever com 100% de certeza se esse programa vai acabar até o fim (parar) ou se ficará travado num loop infinito.

• Contradição

- * Imagine que construíram um algoritmo H P/ o halting Problem c/ 2 entradas: um Programa P e dados de entrada X desse Programa.

- H dá uma resposta definitiva.



- é criada uma máquina D , c/ H como peça interna, que receberá apenas um Programa P como entrada, passa P e H testar o Programa contra ele mesmo (código recebendo seu próprio código como entrada)

- D é programada p/ fazer o resto do que H disser.



- e se entrada de D for D:

- ▲ H analisa $D(D)$ e diz "Para":

- Se H diz "Para", a estrutura externa D entra num loop $\rightarrow H$ v  assumir corretamente

- ▲ H analisa $D(D)$ e diz "loop":

- se H diz "loop", a estrutura externa de D para imediatamente $\rightarrow H$ v's assumiu corretamente.

↳ Passa código de D #1 Danular, H chega numa conclusão, mas como D inverte no final, a previsão de D sempre estará errada.

Si $\tilde{n}$ existe

5. Diagonalização de Cantor: a estrutura de prova do halting problem é uma aplicação direta da diagonalização

- matriz CI: linhas - todas as MT's possíveis e colunas - todas entradas possíveis

- a célula (i, j) é 1 se M_i para $cl w_j$, e 0 se entra em loop

- a máquina D representa a diagonal principal, onde a máquina testa ela mesma

- ↳ D inverte todo bit da diagonal.

- ▲ Logo, D é diferente de todas as linhas em pelo menos em um ponto (diagonal principal).

$MT_1, MT_2, \dots, MT_n$ $\rightarrow$ in jede ser. nennung des MT's fassweis, bis tam Reloments um, valor, diferente

w_1	1	-	-	-	0	-	-	-	-
w_2	-	0	1	-	-	1	-	-	-
$\vdots$	-	-	-	-	1	-	-	-	-
w_m	-	-	-	1	-	0	-	-	-

- ▲ Cantor usou $\mathbb{P}$ para provar que a lista dele estava incompleta; Turing usou $\mathbb{P}$ para provar que se D não bate com nenhuma MT , D não existe; e D não existe, H também não existe.

↳ Se M existisse, nada impediria D de existir

9. Há linguagens não computáveis. Como é possível demonstrar sua existência?

8 Conjuntos: $\cdot \Sigma^*$ é um conjunto infinito numerável

- L é o conjunto de todas as linguagens passíveis $\rightarrow$ partes de Σ^*

↳ Se um conjunto é IE (Σ^*), seu conjunto das partes é INE (teorema de Cantor) $P(\Sigma^*)$

- Consenso de todas MT's é IE

Como o conjunto das MI's é IE e de todas as linguagens é JUE, tentar embelezhar uma linguagem em uma máquina causaria uma impossibilidade matemática - existem mais linguagens que máquinas, logo a maioria das linguagens existentes é não-computável.

• exemplo prático: halting problem

- $A_{\text{tim}} = \{ \langle M, w \rangle \mid M \text{ é uma MT que aceita } w \}$.

- uma linguagem só é ^{decidível} computável se existe uma MT que a decida

- *N. existe uma MT que resolve o halting problem, logo Atm n. é computável.